

Deep Dive — Module 8: JWT (Authentication & Authorization)

***Goal:** understand how the system knows who you are and what you may do — the structure of a JWT, the login flow, how the token travels on every request, role-based access control, expiry, and logout/revocation.*

Where it sits: JWT is the **security wrapper** around the whole backend. After login the server hands the user a signed token; every subsequent API call carries it, and Spring Security validates it before any controller runs. This is how clerks, finance managers, and admins get different access to the same endpoints.

1. What a JWT is (and why it's stateless)

A **JWT (JSON Web Token)** is a signed string the server issues at login. The client sends it with each request to prove identity. Because the token itself *carries* the user id, role, and expiry — and is cryptographically signed — the server can trust it **without a database lookup per request**. That statelessness is what makes JWT fit a REST API: any backend instance can validate any token using just the shared secret.

2. Anatomy: three parts

A JWT is three base64url segments joined by dots: `header.payload.signature`.

```
Header:    { "alg": "HS256", "typ": "JWT" }
Payload:   { "sub": "jsmith", "userId": "usr-0042",
             "role": "ROLE_FINANCE_MANAGER", "department": "procurement",
             "iat": 1748563200, "exp": 1748649600 }
Signature: HMACSHA256( base64(header) + "." + base64(payload), SECRET_KEY )
```

- **Header** — algorithm + type.
- **Payload** — the *claims* (who, role, issued-at `iat`, expiry `exp`). **Not secret** — anyone can decode it; never put passwords or secrets here.

- **Signature** — proves the token was issued by *this* server and hasn't been altered. Change one byte of the payload and the signature no longer matches.

HS256 (symmetric, one shared secret) is the local default; the project notes RS256 (asymmetric, public/private keys) for production so verifiers don't need the signing secret.

3. Login flow

1. React → POST /api/auth/login { "username": "jsmith", "password": "!. " }
2. Backend→ load user from PostgreSQL, verify the bcrypt password hash
3. Backend→ build JWT (userId, role, department, exp), sign with the secret
4. Backend→ { "token": "eyJ.", "expiresIn": 86400, "role": "FINANCE_MANAGER" }
5. React → store the token (in memory / sessionStorage)

Passwords are stored as **bcrypt hashes**, never plaintext; login compares the hash. The login endpoint itself is rate-limited (5/min per IP) to slow brute-force attempts.

4. Using the token on every request

```
GET /api/documents
```

```
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.
```

```
JwtAuthenticationFilter:
```

- extract token from the Authorization header
- verify the signature (was it really issued by us?)
- check exp (still valid?)
- extract role from the payload
- set Spring Security context
- allow the request to reach the controller

The filter runs **before** controllers (`OncePerRequestFilter`), so by the time your method executes, the authenticated user and role are known.

5. Role-based access control (RBAC)

Three roles with escalating permissions:

```
ROLE_CLERK           → upload, search, view own department's documents
ROLE_FINANCE_MANAGER → all clerk rights + view all departments + manage alerts
ROLE_ADMIN           → all rights + delete documents + view audit logs + manage users
```

Enforced declaratively in Spring Security:

```
@DeleteMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')") // only ADMIN
public ResponseEntity<Void> delete(@PathVariable UUID id) {.. }

@GetMapping("/alerts")
@PreAuthorize("hasAnyRole('FINANCE_MANAGER', 'ADMIN')") // managers + admins
public ResponseEntity<List<AlertDTO>> getAlerts() {.. }
```

```
DELETE /api/documents/{id}    clerk → 403    admin → 200
GET    /api/alerts           clerk → 403    manager/admin → 200
```

Two important nuances: (1) **row-level** filtering (clerks see only *their department*) happens in the service layer, beyond what a role annotation expresses; (2) RBAC must be enforced on the **backend** — hiding buttons in React is UX, not security.

6. Expiry, logout & revocation

JWTs are stateless, which creates a known tension: a signed token is valid until it expires, even if you "log out." The project handles this with:

```
Lifetime: 24 hours (configurable in application.yml)
Expired token → JwtFilter sees exp passed → 401 Unauthorized

Logout: POST /api/auth/logout
  → server adds the token to an in-memory (or Redis) blocklist
  → the filter rejects blocklisted tokens even before they expire
```

So revocation reintroduces a *small* piece of server state (the blocklist) precisely to solve "log me out now." Short expiry limits the damage window if a token leaks.

7. Security considerations (project-specific)

- **Secret management:** the signing secret comes from an env var / vault, never source or images. A leaked secret lets anyone forge tokens.
 - **Transport:** tokens travel over TLS only; never in URLs (they'd land in logs/history).
 - **Storage on the client:** in memory / `sessionStorage`, not `localStorage`, to reduce XSS theft risk.
 - **Claims are public:** the payload is readable; signature provides *integrity*, not *confidentiality*.
 - **Audit:** `LOGIN`, `LOGIN_FAILED`, and `LOGOUT` are written to the append-only audit log (Module 1).
-

8. Common pitfalls

- **Putting secrets in the payload** — it's only base64, not encrypted; anyone can read it.
 - **Frontend-only RBAC** — always enforce roles server-side.
 - **Long-lived tokens with no revocation** — a leaked token stays valid; use short expiry + blocklist.
 - **Storing tokens in `localStorage`** — broader XSS exposure than in-memory/`sessionStorage`.
 - **Weak/committed signing secret** — enables forged tokens; keep it strong and external.
 - **Forgetting to check `exp`** — accepting expired tokens.
 - **Plaintext passwords** — always bcrypt; compare hashes.
-

9. Practice exercises

Use [jwt.io] mentally (decode by hand) and a small Spring Security app.

1. Take a sample JWT and decode the header and payload by base64 — read the claims. Note you did this *without* the secret.
2. Change one character in the payload and explain why the signature check now fails.

3. Implement `/api/auth/login` : verify a bcrypt hash, then issue a signed JWT with role + 24h expiry.
 4. Write a `JwtAuthenticationFilter` that validates signature + expiry and sets the security context.
 5. Add `@PreAuthorize` so only ADMIN can delete and only MANAGER/ADMIN can read alerts; test each role → expect 200/403.
 6. Add service-layer filtering so a clerk's `GET /documents` returns only their department.
 7. Implement logout with an in-memory blocklist; confirm a logged-out (but unexpired) token is rejected.
 8. Reduce expiry to 60 seconds, wait, and confirm a 401 with `AUTH_FAILED` .
-

10. Self-check questions

- Why is JWT "stateless," and what advantage does that give a REST API?
 - What are the three parts of a JWT, and what does the signature guarantee?
 - Why must you never store secrets in the payload?
 - Where is RBAC enforced, and why isn't hiding UI enough?
 - How does the system filter results by department, beyond role checks?
 - Why does logout require a blocklist, and what does that imply about statelessness?
 - Why short expiry plus revocation rather than one or the other?
 - How are passwords stored and verified?
-

11. Glossary

- **JWT** — signed JSON token carrying identity claims.
- **Claim** — a field in the payload (`role` , `exp` , etc.).
- **Signature** — HMAC/RSA proof of integrity and origin.
- **HS256 / RS256** — symmetric / asymmetric signing algorithms.
- **bcrypt** — slow password hashing function.
- **RBAC** — Role-Based Access Control.
- `@PreAuthorize` — Spring method-level role check.

- **Blocklist** — set of revoked tokens rejected before expiry.
 - **exp** / **iat** — expiry / issued-at timestamps.
-

Navigation: [← Module 7 Tesseract OCR](#) | [Index](#) | [Module 9 PDFBox & POI →](#)